Lezione 7 – Ereditarietà e polimorfismo

`extends`, `super`, binding dinamico, classi astratte

Mattia Fonisto

Laboratorio di Programmazione – Ingegneria Informatica, II anno
Università degli Studi di Napoli Federico II – Sede di San Giovanni a Teduccio

27 ottobre 2026

Agenda della lezione (2h)

1. **Ripasso e Homework 6** (*15 min*)
   - uno di voi alla lavagna con `Segment`, `Frazione` o `Misure`: lo commentiamo insieme
2. **Ereditarietà** (*25 min*)
   - la relazione *è-un*; `extends`, `super` e i costruttori
   - che cosa vede la sottoclasse: `private` e `protected`; ridefinire un metodo, `@Override`
3. **Polimorfismo** (*20 min*)
   - upcasting e binding dinamico; array polimorfici; downcasting
   - classi e metodi astratti
4. **Esercitazione** – tutti insieme (*50 min*)
   - *Commentiamo insieme*: `Forma`, `Cerchio`, `Rettangolo` – un array di forme
   - *Insieme, passo passo*: `Dipendente` e `Manager`
5. **Wrap-up e Homework 7** (*10 min*)

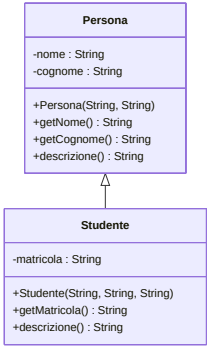
Ereditarietà: la relazione *è-un*

Ereditarietà

Una classe nuova, la *sottoclasse*, si definisce a partire da una classe esistente, la *superclasse*: ne eredita variabili di istanza e metodi, e ne aggiunge di propri o ne ridefinisce alcuni.

È la generalizzazione-specializzazione della lezione 5: *uno studente è una persona*. Tutto ciò che vale per una persona (nome, cognome, descrizione) vale per uno studente, che in più ha una matricola.

- **è-un**: un oggetto `Studente` è anche un oggetto `Persona`, e si può usare dovunque serva una `Persona`: è il polimorfismo
- **riuso**: `Studente` non riscrive `nome`, `cognome`, `getNome`, `getCognome`: li eredita
- **gerarchia**: dal generale al particolare, anche su più livelli (borsista, studente, persona)



In UML la freccia con il triangolo vuoto va dalla sottoclasse alla superclasse (lezione 5). I membri ereditati non si ripetono; `descrizione` compare in entrambe perché `Studente` la *ridefinisce*.

Ereditarietà in Java: `extends` – `Persona.java` e `Studente.java`

```java
/** Una persona: nome e cognome. */
public class Persona {

    private final String nome;
    private final String cognome;

    public Persona(String nome,
                   String cognome) {
        this.nome = nome;
        this.cognome = cognome;
    }

    public String getNome() {
        return nome;
    }

    public String getCognome() {
        return cognome;
    }

    public String descrizione() {
        return nome + " " + cognome;
    }
}
```

```java
/** Uno studente E' UNA persona con matricola. */
public class Studente extends Persona {

    private final String matricola;

    public Studente(String nome,
                    String cognome,
                    String matricola) {
        super(nome, cognome);
        this.matricola = matricola;
    }

    public String getMatricola() {
        return matricola;
    }
```

- `extends Persona`: `Studente` eredita tutto ciò che `Persona` dichiara; aggiunge `matricola` e `getMatricola`; ridefinisce `descrizione` (fra due slide)
- `super(nome, cognome)` chiama il costruttore di `Persona`: prossima slide. `Persona` non è stata toccata (e non è quella della lezione 6)

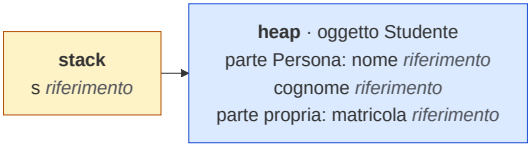
Costruire un oggetto derivato: `super(...)` – `ProvaPersone.java`

```
Persona p = new Persona("Anna", "Russo");
Studente s = new Studente("Luca", "Bianchi",
        "N86001234");

System.out.println(p.descrizione());
System.out.println(s.getNome());
System.out.println(s.getMatricola());
System.out.println(s.descrizione());

// p.getMatricola();   // ERRORE
```

```
Anna Russo
Luca
N86001234
Luca Bianchi, matricola N86001234
```



Un oggetto `Studente` contiene **tutte** le variabili di istanza di `Persona`, più le proprie: una "parte `Persona`" e una parte propria. La parte `Persona` la sa inizializzare solo il costruttore di `Persona`.

- `super(nome, cognome)` lo chiama: di norma è la **prima istruzione** del costruttore; poi `Studente` inizializza la parte propria. È l'analogo di `this(...)` (lezione 6), che chiama un costruttore della *stessa* classe
- senza `super(...)` né `this(...)`, il compilatore inserisce `super()`: se la superclasse non ha un costruttore senza argomenti, errore di compilazione

`p.getMatricola()` non compila: le persone non hanno matricola. L'ereditarietà va in una direzione sola.

Che cosa vede la sottoclasse: `private` e `protected`

In `Studente` (versione 1):

```
@Override
public String descrizione() {
    // return nome + " " + cognome + ...
    // ERRORE: nome e' private in Persona
    return super.descrizione()
            + ", matricola " + matricola;
}
```

`nome` fa parte di ogni `Studente`, ma è `private` in `Persona`: visibile solo in `Persona` (lezione 5). `Studente` lo legge con i metodi `public`: `getNome()`, o `super.descrizione()`.
In `Persona` e `Studente` (versione 2):

```
protected final String nome;
protected final String cognome;
```

```
// lecito: nome e cognome sono protected
return nome + " " + cognome
        + ", matricola " + matricola;
```

Modificatore	Visibile in
`private`	la classe che lo dichiara
`protected`	la classe, le sottoclassi, il package (lezione 9)
`public`	tutte le classi

- `protected` apre la variabile alle sottoclassi e alle classi della stessa cartella (stesso package: lezione 9): nella versione 2 anche `ProvaPersone` vede `s.nome`; un client in un'altra cartella no
- l'ereditarietà **non aggira l'information hiding**: le variabili `private` di `Persona` stanno in ogni `Studente`, che però non le vede
- `protected` ha un costo: se la variabile non è `final`, la sottoclasse può assegnarle un valore che la superclasse non ammette. Con `private` e i `get`, le sottoclassi dipendono solo dall'interfaccia pubblica di `Persona`: se `Persona` rinomina una variabile, non cambiano

Ridefinire un metodo: *overriding* e `@Override`

In `Persona`:

```
    public String descrizione() {
        return nome + " " + cognome;
    }
```

In `Studente`:

```
    @Override
    public String descrizione() {
        return super.descrizione()
                + ", matricola " + matricola;
    }
```

In `ProvaPersone`:

```
System.out.println(p.descrizione());
System.out.println(s.descrizione());
```

```
Anna Russo
Luca Bianchi, matricola N86001234
```

Ridefinizione (*overriding*)

La sottoclasse riscrive un metodo della superclasse con la **stessa firma** e lo **stesso tipo di risultato**, ma un altro corpo: sugli oggetti della sottoclasse vale la nuova versione.

- `super.descrizione()` chiama la versione di `Persona` sullo stesso oggetto: la nuova versione **estende** la vecchia. Senza `super`, `descrizione()` chiamerebbe se stessa all'infinito
- `@Override` è un'*annotazione*: "questo metodo ne ridefinisce uno della superclasse". Se non è vero, per esempio per un errore di battitura (`descrizone`), il compilatore lo segnala; senza l'annotazione avremmo un metodo nuovo e nessun avviso
- non è l'**overloading** della lezione 3: lì stesso nome e parametri *diversi* nella stessa classe; qui stessi parametri in due classi diverse

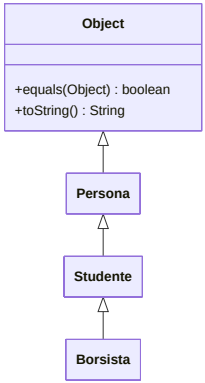
Ereditarietà in Java: le regole

Che cosa si eredita e che cosa no

- si ereditano variabili di istanza e metodi non `private`; quelli `private` fanno parte dell'oggetto, ma la sottoclasse non li vede
- **non** si ereditano i costruttori: ogni classe scrive i suoi, e chiama quelli della superclasse con `super(...)`
- i membri `static` appartengono alla classe: si usano con il suo nome e non partecipano al binding dinamico (prossime slide)
- un metodo `final` non si può ridefinire; una classe `final` non si può estendere (`String` lo è). È il terzo uso di `final`, dopo le variabili locali (lezione 2) e di istanza (lezione 6): sempre "non cambia più"

La forma della gerarchia

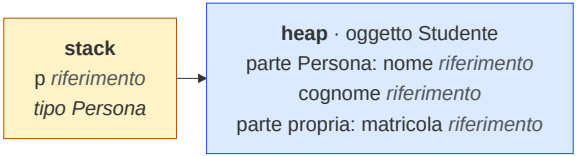
- una classe ha **una sola** superclasse diretta: niente ereditarietà multipla (le interfacce, lezione 9, ne offrono una parte)
- è transitiva: `Borsista extends Studente` eredita anche ciò che `Studente` ha ereditato da `Persona`
- una classe senza `extends` estende `Object`: ogni classe ne discende e ne eredita `equals`, `toString` e altri metodi. Per questo, come detto nella lezione 6, `equals` compila anche su una classe che non lo definisce: lezione 8



`Borsista` sarà uno degli homework. `Object` sta in `java.lang`, come `String`.

Polimorfismo: *upcasting* – `Polimorfismo.java`

```
Persona p = new Studente("Luca", "Bianchi",
        "N86001234");   // upcasting
System.out.println(p.getNome());
System.out.println(p.descrizione());
// p.getMatricola();   // ERRORE
```



Il riferimento è di tipo `Persona`; l'oggetto a cui porta è uno `Studente`. I due tipi non coincidono, e va bene così: il riferimento copiato in `p` (lezione 6) ha un tipo più generale di quello dell'oggetto.

Upcasting

Un riferimento di tipo superclasse può portare a un oggetto di una qualunque sottoclasse: uno `Studente` è *una* `Persona`. La conversione è **implicita**, come `int` → `double` nella lezione 4: vince il tipo più generale.

- quali metodi si possono chiamare su `p`? Li decide il **compilatore** dal **tipo del riferimento**: quelli di `Persona`. `p.getNome()` compila, `p.getMatricola()` no, anche se l'oggetto ce l'ha
- `p.descrizione()` compila: `Persona` ha `descrizione`. Ma ne esistono due versioni, di `Persona` e di `Studente`: quale viene eseguita?
- l'oggetto non cambia: resta uno `Studente`. Cambia solo ciò che `p` permette di vedere

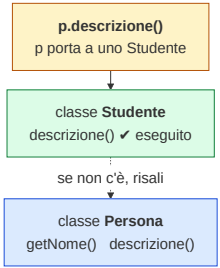
Polimorfismo: binding dinamico

```
Luca
Luca Bianchi, matricola N86001234
```

`p.descrizione()` ha eseguito la versione di `Studente`: la classe dell'**oggetto**, non del riferimento.

Polimorfismo

La stessa chiamata, `x.descrizione()`, esegue codice diverso a seconda della classe dell'oggetto a cui `x` porta in quel momento. La scelta avviene **a tempo di esecuzione** (*binding dinamico*): la fa la JVM, non il compilatore.



È il terzo principio della lezione 5, `f.area()` su cerchi e rettangoli: lo faremo nell'esercitazione. Due momenti, due controlli:

- **compilazione**: `p` è `Persona`; `Persona` ha `descrizione()`? Sì: la chiamata è lecita
- **esecuzione**: `p` porta a uno `Studente`: la JVM cerca `descrizione()` da `Studente`

La JVM parte dalla classe dell'oggetto e risale finché trova il metodo. Per `p.getNome()` in `Studente` non c'è nulla: si risale a `Persona`.

Vale per i metodi che si possono ridefinire. Per i metodi `static`, `private` e `final`, per i costruttori e per la scelta fra metodi in overloading (lezione 3) decide il compilatore: *binding statico*.

Polimorfismo: un array di `Persona` – `Polimorfismo.java`

```java
/** LEGGE la persona ricevuta e la stampa. */
public static void stampa(Persona x) {
    System.out.println(x.descrizione());
}
```

Nel `main`:

```java
Persona[] elenco = {
    new Persona("Anna", "Russo"),
    new Studente("Luca", "Bianchi", "N86001234"),
    new Persona("Marco", "Esposito"),
    new Studente("Sara", "Greco", "N86005678")
};
for (Persona x : elenco) {
    stampa(x);
}
```

```
Anna Russo
Luca Bianchi, matricola N86001234
Marco Esposito
Sara Greco, matricola N86005678
```

- un `Persona[]` può contenere riferimenti a persone e a studenti: ogni elemento è un upcasting. È un **array polimorfico**
- il ciclo e `stampa` sono scritti una volta sola, in termini di `Persona`; a ogni giro `x.descrizione()` esegue la versione giusta per l'oggetto di quel giro
- anche passare `x` a `stampa` è un upcasting: accetta una `Persona`, quindi anche uno `Studente`

Che cosa si guadagna. Se domani aggiungiamo `Docente extends Persona`, né `stampa` né il ciclo cambiano: funzionano anche per i docenti. Il codice che usa la gerarchia è scritto per la superclasse e non conosce l'elenco delle sottoclassi.

Polimorfismo: *downcasting* – `Downcast.java`

```
Persona p = new Studente("Luca", "Bianchi",
        "N86001234");
Persona q = new Persona("Anna", "Russo");

// Studente s = p;   // ERRORE: serve il cast
Studente s = (Studente) p;   // ok: Studente
System.out.println(s.getMatricola());

Studente t = (Studente) q;   // compila, ma...
System.out.println(t.getMatricola());
```

```
N86001234
Exception in thread "main"
  java.lang.ClassCastException: class Persona
  cannot be cast to class Studente ...
```

Il programma termina su `(Studente) q`: la JVM ha controllato che l'oggetto di `q` non è uno `Studente`. Il messaggio è un'*eccezione*, lezione 10.

Downcasting

Assegnare a un riferimento di tipo sottoclasse un riferimento di tipo superclasse. Serve il **cast** `(Studente) p`: compila perché `Studente` è sottoclasse di `Persona`, ma riesce solo se l'oggetto è uno `Studente`.

	Primitivi (L4)	**Classi**
implicito	`double d = i;`	`Persona p = s;`
esplicito	`int i = (int) d;`	`Studente s = (Studente) p;`
rischio	perde i decimali	errore a tempo di esecuzione

- per sapere *prima* se l'oggetto è uno `Studente`, Java ha un operatore apposta (lezione 8): fino ad allora, il cast si fa solo se si è certi della classe dell'oggetto

Classi e metodi astratti

In `Forma`, la superclasse dell'esercitazione:

```
public abstract class Forma {
    private final String colore;

    public Forma(String colore) {
        this.colore = colore;
    }

    public String getColore() {
        return colore;
    }

    public abstract double area();

    public boolean piuGrandeDi(Forma altra) {
        return area() > altra.area();
    }
```

In `ProvaForme`: riferimenti e array di tipo `Forma` sono leciti

```
// Forma f = new Forma("nero");   // ERRORE
Forma[] forme = {
    new Cerchio("rosso", 1),
```

Classe astratta, metodo astratto

Un metodo `abstract` ha la firma ma non il corpo: dice che cosa ogni sottoclasse deve saper fare. Una classe con un metodo astratto è `abstract`: si può estendere e usare come tipo di riferimenti, ma **non creare oggetti**.

- una sottoclasse **deve** ridefinire tutti i metodi astratti che eredita, altrimenti è a sua volta `abstract`
- `piuGrandeDi` ha il corpo e chiama `area()` su `this` e su `altra`: a tempo di esecuzione gli oggetti sono cerchi e rettangoli, e la JVM esegue la loro `area`. Un metodo della superclasse può usare un metodo che ha corpo solo nelle sottoclassi
- `Forma` ha anche la parte comune: `colore`, il costruttore (chiamato dalle sottoclassi con `super(colore)`), `getColore`. In UML i nomi astratti sono in *corsivo*

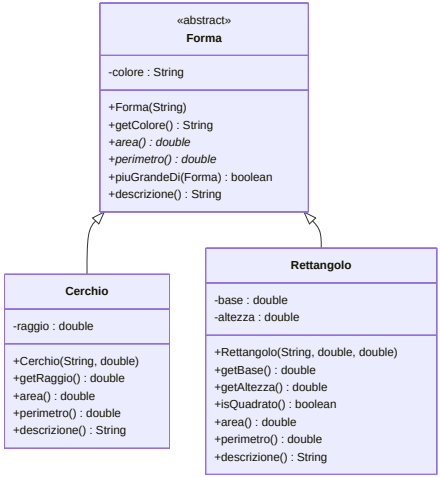
Commentiamo insieme (20 min) – il problema

Il problema. Un programma di disegno tiene in un array le forme sul foglio: cerchi e rettangoli, ciascuno con un colore. Deve stampare la descrizione di ogni forma, calcolare l'area totale e trovare la forma più grande.

Le regole

- ogni forma ha un colore, un'area, un perimetro e una descrizione
- non esiste una forma "generica": sul foglio ci sono solo cerchi e rettangoli. Domani, forse, anche triangoli
- un cerchio ha un raggio, un rettangolo una base e un'altezza: un valore non positivo diventa 1. Dopo la creazione una forma non cambia più
- un rettangolo sa dire se è un quadrato
- di due forme si può dire quale ha l'area maggiore, qualunque forme siano

La soluzione è già scritta: la classe astratta `Forma`, le sottoclassi `Cerchio` e `Rettangolo` e il programma `ProvaForme`. Li leggiamo insieme, riga per riga, cercando `extends`, `super`, `@Override` e `abstract`.



Commentiamo insieme – `Forma.java`

```java
/** Una forma piana, con un colore. Astratta:
 *  esistono solo cerchi, rettangoli... */
public abstract class Forma {

    private final String colore;

    public Forma(String colore) {
        this.colore = colore;
    }

    public String getColore() {
        return colore;
    }

    /** Ogni forma la calcola a modo suo. */
    public abstract double area();

    public abstract double perimetro();

    /** true se ha area maggiore di altra. */
    public boolean piuGrandeDi(Forma altra) {
        return area() > altra.area();
    }

    public String descrizione() {
        return "forma " + colore;
    }
}
```

- `colore` è comune a tutte le forme: sta qui, una volta sola, `private final`
- il costruttore non si chiama mai con `new Forma(...)`: lo chiamano `Cerchio` e `Rettangolo` con `super(colore)`
- `area` e `perimetro` terminano con `;`: niente corpo. Ogni sottoclasse deve scriverli
- `piuGrandeDi` e `descrizione` hanno il corpo e si ereditano: `descrizione` è minima e le sottoclassi la ridefiniranno

Commentiamo insieme – Cerchio.java e Rettangolo.java

```java
public class Cerchio extends Forma {

    private final double raggio;

    public Cerchio(String colore,
                   double raggio) {
        super(colore);
        if (raggio > 0) {
            this.raggio = raggio;
        } else {
            this.raggio = 1;
        }
    }

    @Override
    public double area() {
        return Math.PI * raggio * raggio;
    }

    @Override
    public String descrizione() {
        return "cerchio " + getColore()
                + ", raggio " + raggio;
    }
```

```java
public class Rettangolo extends Forma {

    private final double base;
    private final double altezza;

    public boolean isQuadrato() {
        return base == altezza;
    }

    @Override
    public double area() {
        return base * altezza;
    }
```

- `super(colore)` per primo; poi il controllo sul raggio (non positivo: 1), come in `Circle` della lezione 6. Il costruttore di `Rettangolo`, non mostrato, controlla allo stesso modo `base` e `altezza`
- `area` e `perimetro` danno corpo ai metodi astratti: senza uno dei due, `Cerchio` non compila (dovrebbe essere `abstract`). `descrizione` usa `getColore()`: `colore` è `private`. `isQuadrato` esiste solo in `Rettangolo`

Commentiamo insieme – ProvaForme.java (1/2)

```java
/** LEGGE l'array e somma le aree. */
public static double areaTotale(Forma[] forme) {
    double totale = 0;
    for (Forma f : forme) {
        totale += f.area();
    }
    return totale;
}

/** LEGGE l'array e restituisce la forma
 *  piu' grande (precondizione: almeno una). */
public static Forma piuGrande(Forma[] forme) {
    Forma max = forme[0];
    for (Forma f : forme) {
        if (f.piuGrandeDi(max)) {
            max = f;
        }
    }
    return max;
}
```

Da notare, leggendo:

- `f` è un riferimento di tipo `Forma`; a ogni giro l'oggetto è un `Cerchio` o un `Rettangolo`. `f.area()` compila perché `Forma` dichiara `area`; esegue la versione della classe dell'oggetto
- `areaTotale` non nomina mai `Cerchio` né `Rettangolo`: funzionerà anche per il `Triangolo` dell'homework, senza cambiare una riga
- `piuGrandeDi` è scritto in `Forma`, ma quando viene eseguito `this` e `altra` portano a oggetti concreti: le due chiamate ad `area()` sono entrambe polimorfiche
- `piuGrande` restituisce un `Forma`: il client che la riceve non sa se è un cerchio o un rettangolo, e per stamparla non ha bisogno di saperlo

Commentiamo insieme – ProvaForme.java (2/2)

```java
public static void main(String[] args) {
    Forma[] forme = {
        new Cerchio("rosso", 1),
        new Rettangolo("blu", 3, 4),
        new Cerchio("verde", 2),
        new Rettangolo("giallo", 2, 2)
    };
    for (Forma f : forme) {
        System.out.printf("%s: area %.2f%n",
                f.descrizione(), f.area());
    }
    System.out.printf("area totale: %.2f%n",
            areaTotale(forme));
    Forma g = piuGrande(forme);
    System.out.println(g.descrizione());

    // Forma f = new Forma("nero");   // ERRORE
    // Rettangolo r = forme[1];   // ERRORE: cast
    Rettangolo r = (Rettangolo) forme[1];
    System.out.println(r.isQuadrato());
}
```

```
cerchio rosso, raggio 1.0: area 3.14
rettangolo blu 3.0 x 4.0: area 12.00
cerchio verde, raggio 2.0: area 12.57
rettangolo giallo 2.0 x 2.0: area 4.00
area totale: 31.71
cerchio verde, raggio 2.0
false
```

Rispondiamo insieme:

1. perché `f.descrizione()` non stampa “forma rosso”?
2. quante versioni di area esistono? Quante di `piuGrandeDi`?
3. senza `perimetro` in `Cerchio`, quale file non compila?
4. perché `r.isQuadrato()` vuole il cast, e `f.area()` no?

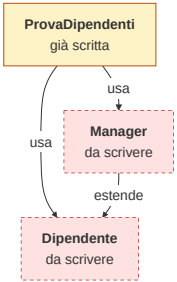
Insieme, passo passo (30 min) – il problema

Il problema. Un'azienda tiene il personale in un array e deve calcolare il costo mensile degli stipendi. Ci sono dipendenti e manager: un manager è un dipendente con in più un bonus mensile.

Le regole

- ogni dipendente ha un nome e uno stipendio base mensile, positivo (precondizione); lo stipendio di un dipendente è la base
- un aumento del p% fa crescere la base del p%; il nome non cambia mai
- un manager ha anche un bonus mensile fisso: il suo stipendio è base più bonus. L'aumento riguarda solo la base
- di ogni dipendente si può avere una descrizione: nome e stipendio; per un manager anche il bonus
- il costo mensile dell'azienda è la somma degli stipendi di tutto il personale

`ProvaDipendenti` è già scritto: mancano `Dipendente` e `Manager`. Scriviamo prima la superclasse, poi la sottoclasse.



`Manager` avrà due ruoli: sarà usata da `ProvaDipendenti` e sarà sottoclasse di `Dipendente`.

Insieme, passo passo – ProvaDipendenti.java

```
public static void main(String[] args) {
    Dipendente a = new Dipendente("Anna", 1800);
    Manager m = new Manager("Luca", 2500, 700);
    Dipendente[] personale = {
        a, m, new Dipendente("Sara", 1600)
    };
    System.out.println(a.descrizione());
    System.out.println(m.descrizione());
    System.out.println(m.getBonus());
    System.out.println(costoMensile(personale));

    for (Dipendente d : personale) {
        d.aumenta(10);
    }
    System.out.println(m.descrizione());
    System.out.println(costoMensile(personale));

    Dipendente d = m;    // upcasting
    System.out.println(d.stipendio());
    // d.getBonus();    // ERRORE: d e' un Dipendente
}
```

Deve stampare:

```
Anna, stipendio 1800.0
Luca, stipendio 3200.0, bonus 700.0
700.0
6600.0
Luca, stipendio 3450.0, bonus 700.0
7190.0
3450.0
```

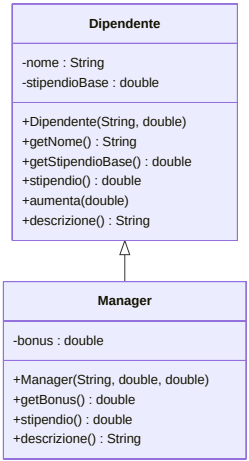
Come nelle lezioni 5 e 6, il client viene prima delle classi: leggendolo ricaviamo l'**interfaccia** di `Dipendente` e di `Manager`. `costoMensile` è un metodo `static` di `ProvaDipendenti`, già scritto come `areaTotale`: somma `d.stipendio()`.

Due indizi: `aumenta(10)` è chiamato su tutti, manager compreso, e il bonus non cambia; `d.stipendio()` stampa 3450, non 2750.

Insieme, passo passo – dal programma di prova alle classi

Prima di scrivere codice, decidiamo insieme:

1. chi è la superclasse? → `Dipendente`: un manager è *un* dipendente, non il contrario
2. quali **variabili di istanza** in `Dipendente`? → `nome`, `final`, e `stipendioBase`, che cambia con `aumenta`: non `final`. `private` entrambe
3. e in `Manager`? → solo `bonus`, `private final`: il resto è ereditato
4. il **costruttore** di `Manager`? → tre parametri: i primi due a `super(...)`, il terzo a `bonus`
5. che cosa ridefinisce `Manager`? → `stipendio`: `super.stipendio() + bonus`. E `descrizione`, che aggiunge il bonus a quella di `Dipendente`
6. `aumenta` va ridefinito? → no: agisce sulla base, che è ereditata
7. serve `protected` perché `Manager` legga la base? → no: basta `super.stipendio()`, o `getStipendioBase()`



Una possibile soluzione (1/2) – `Dipendente.java`

```java
/** Un dipendente: nome e stipendio base. */
public class Dipendente {

    private final String nome;
    private double stipendioBase;   // mensile

    /** Precondizione: stipendioBase > 0. */
    public Dipendente(String nome,
                      double stipendioBase) {
        this.nome = nome;
        this.stipendioBase = stipendioBase;
    }

    public String getNome() {
        return nome;
    }

    public double getStipendioBase() {
        return stipendioBase;
    }

    /** Lo stipendio mensile: la base. */
    public double stipendio() {
        return stipendioBase;
    }

    /** MODIFICA la base: +percentuale %. */
    public void aumenta(double percentuale) {
        stipendioBase += stipendioBase
                * percentuale / 100;
    }

    public String descrizione() {
        return nome + ", stipendio "
                + stipendio();
    }
}
```

- una classe come quelle delle lezioni 5 e 6: non sa che avrà una sottoclasse. Non è astratta: i dipendenti esistono
- `descrizione` chiama `stipendio()` invece di leggere `stipendioBase`: è ciò che farà funzionare la descrizione di un manager

Una possibile soluzione (2/2) – `Manager.java`

```
public class Manager extends Dipendente {
    private final double bonus;    // mensile
    public Manager(String nome,
                   double stipendioBase,
                   double bonus) {
        super(nome, stipendioBase);
        this.bonus = bonus;
    }

    public double getBonus() {
        return bonus;
    }

    @Override
    public double stipendio() {
        return super.stipendio() + bonus;
    }

    @Override
    public String descrizione() {
        return super.descrizione()
                + ", bonus " + bonus;
    }
}
```

- `super(nome, stipendioBase)` per primo: la parte `Dipendente` la inizializza `Dipendente`
- `stipendio` ridefinito **estende** quello di `Dipendente`: `super.stipendio()` è la base, letta senza `protected`
- `m.descrizione()` esegue la versione di `Manager`, che chiama `super.descrizione()`, che chiama `stipendio()`: su un manager è quello di `Manager`, 3200 e non 2500. Il binding dinamico vale anche dentro la superclasse
- `d.aumenta(10)`: il metodo ereditato lavora sulla parte `Dipendente`; `bonus` non è toccato
- `d = m` è un upcasting: `d.stipendio()` stampa 3450. Il tipo di `d` decide che cosa compila (`d.getBonus()` no), l'oggetto che cosa è eseguito

Riepilogo: cosa abbiamo trattato

Concetto (unità)	**Dove compare oggi**
Ereditarietà, relazione *è-un* (U13)	`Studente`, `Cerchio`, `Rettangolo`, `Manager` e le loro superclassi
`extends`, costruttori e `super(...)` (U13)	tutte le sottoclassi di oggi
`private` e `protected` (U13)	`Persona` versioni 1 e 2
Ridefinizione, `super.metodo()`, `@Override` (U13)	`descrizione` in `Studente`; `stipendio` in `Manager`
Che cosa non si eredita; `final`; `Object` (U13)	slide "le regole"
Upcasting e downcasting (U14)	`Polimorfismo`; `Downcast`; `(Rettangolo) forme[1]`; `d = m`
Binding dinamico (U14)	`p.descrizione()`; `f.area()`; `d.stipendio()`
Array polimorfico (U14)	`Persona[]`; `Forma[]`; `Dipendente[]`
Classi e metodi astratti (U14)	`Forma`, `area()` e `perimetro()`

Prossima lezione (L8): la superclasse `Object` – `toString`, `equals(Object)` e `hashCode`, `instanceof`, `clone`; le relazioni tra classi: aggregazione e composizione, ereditarietà o contenimento?

Homework 7 (da svolgere entro la prossima lezione)

Tre sottoclassi, ciascuna con il suo programma di prova (`ProvaTriangolo`, `ProvaContoRisparmio`, `ProvaBorsisti`); le superclassi, già scritte, vanno copiate in `homework/` senza modifiche. Niente `instanceof` né `toString`.

1. `Triangolo.java`: una nuova forma per la gerarchia di oggi (copiate `Forma`, `Cerchio` e `Rettangolo`). Costruttore `Triangolo(String colore, double a, double b, double c)`, con i tre lati: se non formano un triangolo (uno non positivo, o non minore della somma degli altri due) diventano 3, 4, 5; il controllo è un metodo `private static`. `getA()`, `getB()`, `getC()`, `isEquilatero()`, `perimetro()`, `area()` con la formula di Erone $\sqrt{p(p-a)(p-b)(p-c)}$ (p semiperimetro), `descrizione()`. Il programma di prova mette in un `Forma[]` tre triangoli (uno non valido), un cerchio e un rettangolo; stampa descrizione, area e perimetro di ciascuno e l'area totale (con l'`areaTotale` di oggi); infine, con un cast, dice se il secondo triangolo è equilatero.
2. `ContoRisparmio.java`: estende il `ContoBancario` della lezione 6 (copiatelo). Ha un tasso annuo in percentuale e la costante di classe `SALDO_MINIMO`, 100. Costruttore `ContoRisparmio(String intestatario, double primo, double tasso)`; `getTasso()`; `preleva` ridefinito: `false` anche se il prelievo porterebbe il saldo sotto `SALDO_MINIMO`, altrimenti come in `ContoBancario`; `aggiungiInteressi()` versa gli interessi di un anno (saldo per tasso diviso 100). Il programma di prova mette in un `ContoBancario[]` un conto normale e uno di risparmio, entrambi con 500; chiede a ciascuno un prelievo di 450 e poi uno di 300, stampando risultati e saldi; aggiunge gli interessi al conto di risparmio; prova un trasferimento per verso.
3. `Borsista.java`: estende lo `Studente` di oggi (copiate `Persona` e `Studente`). Importo annuo della borsa, `double` positivo; costruttore con quattro parametri; `getImportoBorsa()`; `rataMensile()`, l'importo diviso 12; `descrizione()` ridefinita. Il programma di prova crea un borsista e chiama su di lui un metodo di `Persona`, uno di `Studente` e uno di `Borsista`; mette in un `Persona[]` una persona, uno studente e il borsista e stampa le descrizioni con `stampa(Persona x)`; infine assegna il borsista a un riferimento `Studente` e ne stampa la descrizione.

Domande?

mattia.fonisto@unina.it